
Software Requirements Specification

for

Employee Burnout & Turnover Prediction System

A Hybrid CNN-LSTM Deep Learning Approach

Prepared by R.K.KRISHNI

24BDS1070

17-07-2026

Table of Contents

- Table of Contents
- Revision History
- 1. Introduction
- 1.1 Purpose.....
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions.....
 - 1.4 Product Scope.....
 - 1.5 References
- 2. Overall Description
- 2.1 Product Perspective.....
 - 2.2 Product Functions.....
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints.....
 - 2.6 User Documentation.....
 - 2.7 Assumptions and Dependencies
 - 2.8 Development Process Model - Agile (Scrum)
- 3. External Interface Requirements.....
- 3.1 User Interfaces.....
 - 3.2 Hardware Interfaces
 - 3.3 Software Interfaces.....
 - 3.4 Communications Interfaces
- 4. System Features.....
- 4.1 System Feature 1
 - 4.2 System Feature 2 (and so on).....
- 5. Other Nonfunctional Requirements
- 5.1 Performance Requirements
 - 5.2 Safety Requirements.....
 - 5.3 Security Requirements
 - 5.4 Software Quality Attributes.....
 - 5.5 Business Rules
- 6. Other Requirements.....
- Appendix A: Glossary
- Appendix B: Analysis Models
- Appendix C: To Be Determined List

1. Introduction

1.1 Purpose

The purpose of this project is to develop an AI-based Employee Burnout Prediction System capable of predicting an employee's burnout risk using deep learning techniques. The system uses a Hybrid CNN-LSTM model to analyze employee workload, mental fatigue, work-from-home status, designation, and other organizational factors to estimate burnout probability. The prediction helps HR departments identify high-risk employees and recommend preventive actions.

1.2 Document Conventions

- Python is used as the programming language.
- TensorFlow/Keras is used for deep learning.
- FastAPI is used for backend API development.
- Streamlit is used for frontend visualization.
- PostgreSQL is used as the database.
- Burn Rate is the target variable.
- CNN-LSTM refers to the Hybrid Convolutional Neural Network and Long Short-Term Memory model.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Project Developers
- Machine Learning Engineers
- HR Professionals
- System Testers
- Database Administrators
- End Users (HR Managers)

1.4 Product Scope

The Employee Burnout Prediction System predicts employee burnout levels using historical employee data. It enables HR departments to monitor employee wellbeing, generate risk reports, store prediction history, and support decision-making through an interactive dashboard.

Objectives include:

- Predict burnout risk accurately.
- Identify high-risk employees.
- Reduce employee turnover.
- Assist HR in proactive interventions.

1.5 References

- Kaggle Employee Burnout & Turnover Prediction Dataset
- TensorFlow Documentation
- FastAPI Documentation
- Streamlit Documentation
- PostgreSQL Documentation
- IEEE SRS Standard

2. Overall Description

2.1 Product Perspective

EBTS is a new, self-contained, four-tier application rather than a follow-on to an existing product. It consists of (1) an offline data-preparation and model-training pipeline, (2) a CNN-LSTM inference model persisted to disk, (3) a FastAPI backend that wraps the model behind a REST API, and (4) a Streamlit frontend backed by a PostgreSQL database for prediction logging and monitoring.

2.2 Product Functions

The system shall:

- Load employee dataset.
- Perform data preprocessing.
- Handle missing values.
- Generate synthetic time-series sequences.
- Engineer temporal features.
- Scale numerical features.
- Train Hybrid CNN-LSTM model.
- Predict employee burnout.
- Store prediction history.
- Display prediction dashboard.
- Recommend interventions.
- Provide REST API for predictions.

2.3 User Classes and Characteristics

HR Manager

- Makes predictions
- Views dashboards
- Monitors employees
- Generates reports

Administrator

- Maintains database
- Updates model
- Manages users

Machine Learning Engineer

- Retrains model
- Updates dataset
- Tunes hyperparameters

2.4 Operating Environment

Operating System

- Windows 10/11
- Ubuntu Linux
- macOS

Programming Language

- Python 3.11+

Libraries

- Pandas
- NumPy
- TensorFlow
- Keras
- Scikit-learn
- Matplotlib
- Seaborn
- FastAPI
- Streamlit
- SQLAlchemy
- Psycopg2

Database

- PostgreSQL

2.5 Design and Implementation Constraints

- The model must consume fixed-length sequences (default: 12 months); any change to sequence length requires retraining.
- The FastAPI service must apply the exact same StandardScaler fitted during training to avoid input-distribution mismatch.
- The Kaggle dataset provides only static, per-employee snapshots; because it has no true longitudinal records, temporal sequences are synthetically generated, which is an approximation the team must document as a modeling assumption.
- Development follows an Agile (Scrum) process with fixed-length Sprints, described in Section 2.8; requirements in Section 4 may be re-estimated at Sprint Planning.

2.6 User Documentation

The system provides:

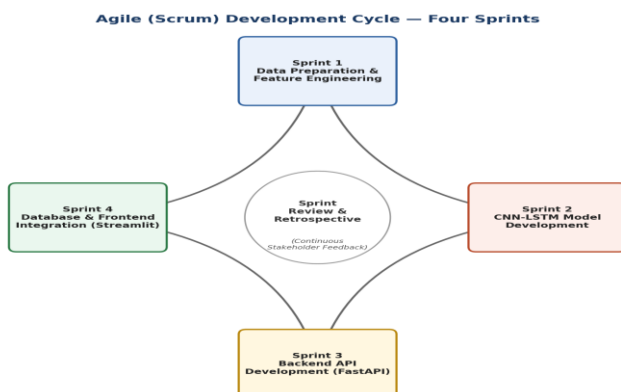
- User Manual
- Installation Guide
- API Documentation
- Swagger Documentation
- Prediction Dashboard Guide

2.7 Assumptions and Dependencies

- Assumes the Kaggle dataset's feature distributions are representative enough of a real organization for the synthetic-sequence approach to yield a usefully predictive model.
- Depends on third-party libraries (TensorFlow/Keras, FastAPI, Streamlit, psycopg2/SQLAlchemy) remaining available and compatible across the project's Python version.
- Assumes HR staff using the dashboard have basic familiarity with the input metrics (e.g., Resource Allocation, Mental Fatigue Score).
- Depends on continued access to sufficient compute (CPU/GPU) for model training during Sprint 2.

2.8 Development Process Model — Agile (Scrum)

The project is executed using an Agile (Scrum) process rather than a single up-front waterfall design. The Product Backlog is organized around the four project phases identified during planning, each delivered as one time-boxed Sprint with its own Sprint Goal, Sprint Review, and Retrospective. This allows the CNN-LSTM model's performance (Section 4.2) to be validated early, with backend and frontend work (Sections 4.3–4.4) proceeding once a usable model artifact exists, while still allowing requirements to be refined between Sprints based on stakeholder feedback.



3. External Interface Requirements

3.1 User Interfaces

The Streamlit dashboard contains:

- Gender dropdown
- Company Type dropdown
- WFH Setup dropdown
- Designation slider
- Resource Allocation slider
- Mental Fatigue slider
- Predict Button
- Risk Score Gauge
- Recommended Actions
- Prediction History

3.2 Hardware Interfaces

Minimum Requirements

- Intel i5 Processor
- 8 GB RAM
- 20 GB Storage

Recommended

- GPU for model training

3.3 Software Interfaces

Interfaces include:

- TensorFlow Model
- FastAPI REST API
- PostgreSQL Database
- Streamlit Frontend
- StandardScaler

3.4 Communications Interfaces

- Frontend-to-backend communication uses synchronous HTTP/REST calls with JSON request and response bodies.
- The FastAPI service exposes interactive documentation (Swagger UI/OpenAPI) at /docs over HTTP(S).
- The backend connects to PostgreSQL over a standard TCP database connection; credentials are supplied via environment variables, not hard-coded.

4. System Features

4.1 Data Preparation Module

Description

Prepares employee data for CNN-LSTM model.

Priority

High

Functional Requirements

REQ-1

Load Kaggle dataset.

REQ-2

Remove Employee ID.

REQ-3

Handle missing values.

REQ-4

Visualize distributions.

REQ-5

Generate synthetic monthly sequences.

REQ-6

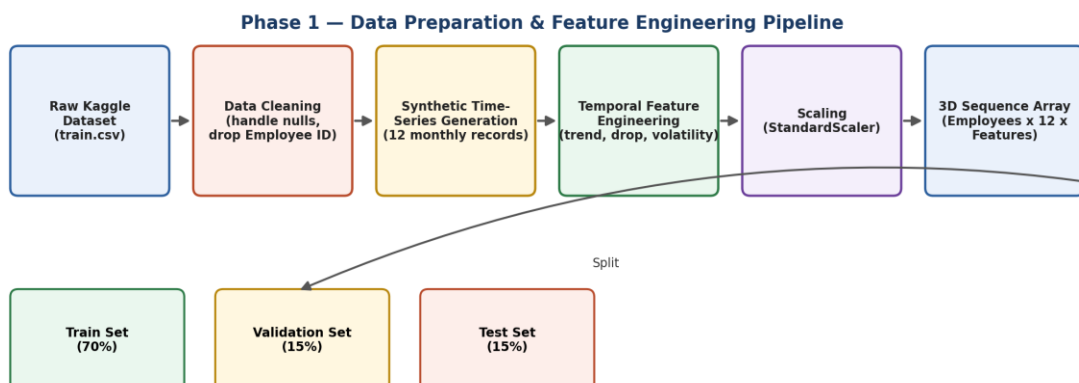
Create temporal features.

REQ-7

Normalize data.

REQ-8

Split into Train/Validation/Test.



4.2 CNN-LSTM Prediction Module

Priority

High

Functional Requirements

REQ-9

Build CNN layers.

REQ-10

Build LSTM layer.

REQ-11

Train Hybrid Model.

REQ-12

Use EarlyStopping.

REQ-13

Save Best Model.

REQ-14

Evaluate Accuracy.

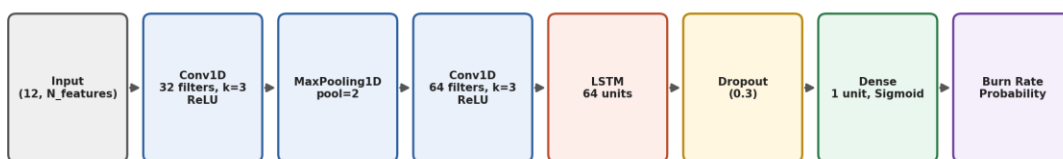
REQ-15

Evaluate ROC-AUC.

REQ-16

Evaluate F1 Score.

CNN-LSTM Hybrid Model Architecture (Phase 2)



CNN layers extract local temporal patterns; LSTM layer models long-term dependencies across the 12-month sequence.

4.3 Prediction API Module

Priority

High

REQ-17

Load saved model.

REQ-18

Load scaler.

REQ-19

Accept employee input.

REQ-20

Preprocess input.

REQ-21

Generate prediction.

REQ-22

Return JSON response.

4.4 Database Module

Priority

Medium

REQ-23

Store prediction history.

REQ-24

Store employee ID.

REQ-25

Store timestamp.

REQ-26

Store risk score.

REQ-27

Store model version.

4.5 Dashboard Module

Priority

High

REQ-28

Provide prediction form.

REQ-29

Display burnout score.

REQ-30

Display intervention recommendations.

REQ-31

Display prediction history.

REQ-32

Display analytics.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The POST /predict endpoint shall return a response within 2 seconds under normal load (single concurrent request) on the target deployment hardware.
- The CNN-LSTM model shall target at least 90% test-set accuracy prior to production deployment, consistent with results reported for comparable CNN-LSTM fatigue/burnout studies.
- The Streamlit dashboard shall render prediction results within 3 seconds of form submission, including network round-trip to the API.

5.2 Software Quality Attributes

- High Accuracy
- Reliability
- Scalability
- Maintainability
- Availability
- Usability
- Portability
- Robustness

5.3 Business Rules

- Every prediction is logged.
- HR users can access reports.
- Only administrators can update the model.
- Burnout risk categories:
 - Low (0–30%)
 - Medium (31–70%)
 - High (71–100%)

6. Other Requirements

Database Schema

Table: predictions

Field	Type
prediction_id	SERIAL
employee_id	VARCHAR
prediction_date	TIMESTAMP
gender	VARCHAR
company_type	VARCHAR
designation	INTEGER
resource_allocation	FLOAT
mental_fatigue	FLOAT
burnout_probability	FLOAT
risk_level	VARCHAR
model_version	VARCHAR

Appendix A: Glossary

- CNN – Convolutional Neural Network
- LSTM – Long Short-Term Memory
- API – Application Programming Interface
- EDA – Exploratory Data Analysis
- WFH – Work From Home
- HR – Human Resources
- ROC-AUC – Receiver Operating Characteristic Area Under Curve
- F1-Score – Harmonic mean of Precision and Recall

Appendix B: Analysis Models

B.2 Data Preparation Pipeline

Shows the flow from raw Kaggle data through cleaning, synthetic sequence generation, temporal feature engineering, scaling, and the train/validation/test split.

B.3 CNN-LSTM Model Architecture

Shows the layer-by-layer structure of the hybrid model, from the input sequence through Conv1D/MaxPooling1D, the LSTM layer, Dropout, and the sigmoid output.

B.4 Agile Sprint Cycle

Shows the four Sprints mapped to the project phases and the continuous Sprint Review/Retrospective cycle that drives iterative refinement of these requirements.

Appendix C: To Be Determined List

- Final model hyperparameters.
- Deployment platform (AWS/Render/Heroku/Local).
- Authentication mechanism.
- Model versioning strategy.
- Notification/alert integration for high-risk employees.